An Introduction & Guide to JALV3

Contents

Chapter 1: Introduction..... 5

Chapter 2: Definitions and Conventions..... 7
Definitions..... 7
Conventions..... 8

Chapter 3: Variables, Constants, Aliases..... 9
Types..... 9
Wide integers (64-bit and 128-bit)..... 10
Arrays..... 11
Multi-dimensional arrays..... 11
Native bit arrays..... 12
Records..... 13
Variables..... 13
Constants..... 15
Unnamed Constants..... 15
Named Constants..... 17
String Literals..... 17
Aliases..... 17

Chapter 4: Strings..... 19
Fixed-capacity strings..... 19
Dynamic strings and the string heap..... 20

Chapter 5: Operators, Casting, Expressions..... 23
Operators..... 23
Casting..... 24
Text conversion and COUNT..... 25
Expressions..... 26

Chapter 6: Flow Control..... 27
BLOCK..... 27
CASE..... 27
FOR..... 28
FOREVER..... 28
IF..... 29
Repeat..... 29
WHILE..... 30

Chapter 7: Other Keywords..... 31
Assert..... 31
INCLUDE..... 31
Message generating..... 31

_DEBUG....31
_ERROR....32
_WARN....32

Chapter 8: Sub-programs: Procedures and Functions....33
Optional parameters and defaults....34

Chapter 9: Pseudo-variables....37

Chapter 10: Pointers....39
Procedure and function pointers....41

Chapter 11: Interrupts....45

Chapter 12: Tasks....47

Chapter 13: Assembly....49
Available Op-codes....50
Common Macros....51
Data Directives....52

Chapter 14: Built-in Functions....53
Multiplication, Division, Modulus Division....53
Floating Point Operations....53
_usec_delay(*cexpr*);....53

Chapter 15: Operator Precedence....55

Chapter 16: Limitations and known issues....57

Chapter 1

Introduction

JAL is a high level language designed to hide the general nuisance of programming a MicroChip PIC processor. It is derived from the original JAL by Wouter van Ooijen, see: Voti

JAL is loosely based on Pascal and is not case sensitive.

This document describes the JAL syntax supported by the JALv3 compiler.

JALv3 is a fork of the JALv2 compiler. In addition to the classic PIC10/12/14/18 targets it supports PIC32 (MIPS32) and ESP32 (Xtensa LX6) devices. It extends the language with pointers, dynamic strings, fixed-capacity strings, native bit arrays, multi-dimensional arrays, wide integers up to 128 bits, optional parameters with defaults, text conversion casts, COUNT, and a cooperative task scheduler. These features are described in the chapters that follow.

This manual is maintained for JALv3 by Matthew Schinkel. The original JALv2 documentation was written by Kyle York and Rob Jansen.

Chapter 2

Definitions and Conventions

Definitions

The following abbreviations are used throughout this guide:

bit

A bit within a byte, 0 <= bit <= 7

Comment

Comments begin with either "--" or ";" and continue through the end of the line.

constant

A numeric constant.

expression

An expression is a sequence of values and operations.Expressions are subdivided into:

cexpr -- constant expression. An expression that can be fully evaluated at compile time. For example 1 + 2.

expr -- any expression. An expression is anything that evaluates to a value, for example: b + c, x + 1, etc.

lexpr -- logical expression. A logical expression. This differs from an expression in that the result is 0 if the expression is zero, and 1 if the expression is anything other than 0.

identifier

Identifies a variable, constant procedure, function, label, etc. Must begin with a letter or '_' followed by any number of of letters (a-z), digits (0-9), or '_'. Note that identifiers beginning with '_' are reserved for the compiler.

program

A program is simply a sequence of statements. Unlike other languages, in JAL, if the execution runs out of statements, the processor will be put to sleep.

scope

Scope is the `visibility' of an identifier. Each statement_block creates a new scope. Anything declared within this scope will not be visible once the scope ends.

A variable can be redefined in a block as follows:

```
VAR BYTE x, z
...
IF (x) THEN
   VAR WORD x, y ; all references to x will refer
               ; to this definition
   ...
```

```
        END IF
        ...
        VAR WORD x ; this is illegal because x already exists
```

statement

A single assignment, definition, control (BLOCK, CASE, IF) or looping (FOR, FOREVER, REPEAT, WHILE).

statement_block

A sequence of statements. Variables, constants, procedures, and functions defined in a statement_block will not be visible outside of the statement_block.

token

The JAL compiler sees only a stream of tokens. An entire program can be written without any line breaks or extra spaces, except of course for comments which are terminated by and end of line.

var – variable

Conventions

The following notational conventions are used throughout this guide:

{ a | b | c } -- one of

must be one of of a,b,c

KEYWORD -- A JALv3 keyword

Upper case denotes a JALv3 keyword

'...' -- literal

Anything between the quotes must be typed exactly.

[...] – optional

Anything between the brackets is optional.

Chapter 3

Variables, Constants, Aliases

Types

The following are the list of types understood by the JALv3 compiler.

Table 1: JALv3 Built-in Types

Type	Description	Range
BIT[1]	1 bit boolean value	0..1
SBIT[1]	1 bit signed value	-1..0
BYTE[1]	8 bit unsigned value	0..255
SBYTE[1]	8 bit signed value	-128..127
WORD	16 bit unsigned value	0..65,535
SWORD	16 bit signed value	-32,768..32,767
DWORD	32 bit unsigned value	0..4,294,967,295
SDWORD	32 bit signed value	-2,147,483,648..2,147,483,647
QWORD	64 bit unsigned value	0..18,446,744,073,709,551,615
SQWORD	64 bit signed value	-2^63..2^63-1
OWORD	128 bit unsigned value	0..2^128-1
SOWORD	128 bit signed value	-2^127..2^127-1
FLOAT[1]	floating point value	+/-10^-44..10^38

[1]base types

The larger types, [S]WORD, [S]DWORD, [S]QWORD, and [S]OWORD are simply derived from the base types using the width specifier. For example, WORD is equivalent to BYTE*2, the later can be used interchangeably with the former. See Wide integers (64-bit and 128-bit) on page 10 for the 64-bit and 128-bit types.

Floating point arithmetic is *very* expensive in terms of both code and data and should best be avoided. JALv3 implements IEEE 754 single precision arithmetic with round-to-nearest-even, denormal values, and the special values infinity and NaN. A floating point value is represented in 4 BYTEs.

A note needs to be made concerning the BIT type. In the original JAL language, the BIT type acted more like a Boolean -- if assigned 0, the value stored would be zero, if assigned any non-zero value, the value stored would be one. This convention is still used in JALv3.

However, JALv3 also understands BIT types more like C bitfields. If, instead of BIT one uses the type BIT*1, the value assigned would be masked appropriately (in other words BIT*1 y = z translates internally to BIT*1 y = (z & 0x0001).

Even though the predefined larger types use standard widths (2 and 4), there is no such requirement imposed by the language. If you need a three byte value, use BYTE*3. The only upper limit is the requirement that any value fit within one data bank.

Finally, BIT and BYTE are distinct, so defining a value of BIT*24 is not the same as defining a value of BYTE*3!

Wide integers (64-bit and 128-bit)

JALv3 extends the integer types beyond the 32-bit DWORD of JALv2 with 64-bit and 128-bit types. They are ordinary integer types: they can be used in expressions, arrays, records, procedure parameters, and as pointer pointee contracts.

Table 2: Wide integer types

Type	Width	Range	Notes
QWORD	8 bytes	0 .. 2^64-1	unsigned 64-bit
SQWORD	8 bytes	-2^63 .. 2^63-1	signed 64-bit
OWORD	16 bytes	0 .. 2^128-1	unsigned 128-bit
SOWORD	16 bytes	-2^127 .. 2^127-1	signed 128-bit

The derivation rule of the smaller types still applies: QWORD is equivalent to BYTE*8, OWORD to BYTE*16, and the signed types add the SIGNED attribute to the same widths.

Literals may be written as decimal numbers (underscores are allowed as digit separators) or in hexadecimal with a 0x prefix. A leading minus sign makes a signed value.

```
CONST QWORD GOLDEN = 0x9E3779B97F4A7C15
VAR QWORD total = 0
VAR SQWORD delta = -1
VAR OWORD counter = 0
VAR SOWORD floor = -1
```

Arithmetic, shifts, bitwise operators, and comparisons operate on the full width of the value. When operands of different widths meet, the narrower value is promoted to the wider type, as with the smaller integer types.

```
total = total + 0x1111111111111111
IF total > GOLDEN THEN
   total = total << 1
END IF
```

A wide value occupies contiguous bytes in data memory, least significant byte first. Byte overlays can be used to inspect or fill the individual bytes:

```
VAR QWORD value
VAR BYTE view[8] AT value
VAR QWORD high AT big_value + 8
```

COUNT returns the storage width in bytes, so COUNT(total) is 8 and COUNT(counter) is 16. See Text conversion and COUNT on page 25 for COUNT and the text conversion casts.

Arrays

JAL allows arrays of any type, including native bit arrays, with one or more dimensions. A one-dimensional array is defined during variable definition using the notation:

```
VAR type '[' cexpr ']' id
```

This defines id as type with *cexpr* elements. These are accessed using brackets. The elements are numbered from zero, so for 5 elements the accessors are 0 to 4.

Example:

```
VAR BYTE stuff[5], xx

xx = 2
stuff[0] = 1
stuff[xx] = 2
xx = stuff[xx]
```

Note: There is no error checking when an array is accessed with a variable. In the above example, if xx is 5 no error will be generated, but the results will not be as expected.

JALv3 adds packed bit arrays (Native bit arrays on page 12) and arrays with more than one dimension (Multi-dimensional arrays on page 11).

Multi-dimensional arrays

JALv3 supports native arrays with more than one dimension. Any number of dimensions may be declared; the elements are stored in row-major order in one contiguous block.

```
VAR BYTE matrix[ROWS][COLS]
VAR BYTE cube[X][Y][Z]
```

Access uses one subscript per dimension, and each dimension is numbered from zero. Constant and runtime subscripts may be mixed freely.

```
VAR BYTE matrix[4][8]
VAR BYTE cube[2][3][4]

matrix[1][2] = 42
VAR BYTE value = cube[1][0][1]
VAR BYTE row = 3
matrix[row][2] = value
```

An initializer may be written as nested lists; the inner list corresponds to the last dimension:

```
VAR BYTE matrix[2][3] = {{1, 2, 3}, {4, 5, 6}}
```

COUNT returns the extent of the first dimension. Applying a subscript first asks for the extent of the next dimension:

```
COUNT(matrix)       -- 2
COUNT(matrix[0])    -- 3
```

Multi-dimensional arrays can have any element type, including BIT and the wide integer types. See Native bit arrays on page 12 for the packed bit-array form.

As with one-dimensional arrays, there is no runtime bounds checking when a variable is used as a subscript. A constant subscript outside the declared range is a compile error.

Native bit arrays

JALv3 stores arrays of BIT natively: one bit per element, packed without padding. A BIT array of N elements occupies (N+7)/8 bytes of data memory.

```
VAR BIT flags[16]
VAR BIT init_flags[4] = {TRUE, FALSE, TRUE, TRUE}
CONST BIT const_flags[4] = {FALSE, TRUE, TRUE, FALSE}
```

Elements are accessed with the ordinary subscript syntax, using either a constant or a runtime index. A BIT value can be tested directly or converted to a byte value:

```
flags[0] = TRUE
flags[idx] = FALSE
VAR BYTE as_byte = BYTE(flags[0])
IF !flags[idx] THEN
   flags[1] = FALSE
END IF
IF flags[0] & flags[2] THEN
   flags[3] = TRUE
END IF
```

Bit arrays may be passed to procedures as fixed-size or open-array formals. An IN OUT formal writes changes back to the caller's array:

```
PROCEDURE read_fixed(BIT IN bits[4]) IS ... END PROCEDURE
PROCEDURE read_open(BIT IN bits[]) IS ... END PROCEDURE
PROCEDURE update(BIT IN OUT bits[17]) IS ... END PROCEDURE
```

Multi-dimensional bit arrays are also supported, for example an 8 by 8 bit grid:

```
VAR BIT grid[8][8]
grid[row][col] = TRUE
```

Multi-bit fields

The packed field type BIT*N declares an N-bit integer (1 to 8 bits) that can be used on its own, as an array element type, or as an overlay on a byte variable. The overlay form places the field at a bit offset inside the target variable, which is the natural way to model hardware register fields.

```
VAR BYTE config_shadow = 0
VAR BIT*3 speed_field AT config_shadow : 0    -- bits 2:0
VAR BIT*2 mode_field  AT config_shadow : 3    -- bits 4:3
VAR BIT*1 enable      AT config_shadow : 5    -- bit 5

speed_field = 5
mode_field = 2
enable = TRUE
VAR BYTE raw = config_shadow                  -- 0x35
```

BIT*N fields may also be arrayed, for example VAR BIT*3 fields[80] for 80 three-bit values. Widths greater than 8 bits are refused with a diagnostic.

Bounds and checking

A constant subscript outside 0 .. N-1 is a compile error (subscript out of range). A runtime index is not range checked, exactly like the other array types, and an open bit-array formal has no compile-time bound inside the callee.

Records

Records are special types, composed of fields which are built-in types, arrays, and/or other records. These are defined with:

```
RECORD identifier IS
   type[*cexpr] id0 [ '[' cexpr ']' ]
   ...
END RECORD
```

Once defined, the RECORD identifier can be use anywhere a simple type can be used. Each individual field is accessed using '.'

Example:

```
RECORD eyeinfo IS
   BYTE left
   BYTE right
END RECORD

;
; a record can be initialized on definition as follows:
;
VAR eyeinfo eye = { 3, 4 }

;
; alternately, each field is accessed with the '.' operator:
;
eye.left = 1
eye.right = 2

;
; A more complex example. This sets eyes[0] to {1,2},
; eyes[1] to {3,4} and eyes[2] to {5,6}:
;
VAR eyeinfo eyes[5] = { { 1, 2 }, { 3, 4 }, { 5, 6 } }

;
; Finally, nested records and arrays are supported
;
RECORD face_r IS
   eyeinfo eyes
   BYTE nose
   BYTE freckels[5]
END RECORD

VAR face_r[5] = {
   { { 1,2 }, 3, {4, 5, 6, 7, 8} },
   { { 2,1 }, 3, {8, 7, 6, 5, 4} }
}
```

Variables

A variable is simply an identifier that holds a value. These identifiers have types associated which define how much space is required to hold the value. The following types are built-in:

The complete format for defining a variable is:

```
VAR [VOLATILE] [SHARED] type[*cexpr] identifier [ '[' [ cexpr ]
```

```
'] ' ]
[ { AT cexpr [ ':' bit ] | var [ ':' bit ] | '{' cexpr1[','
cexpr2...] '}' | IS var } ]
[ '=' cexpr | '{' cexpr1',' ... '}' | '"'...'"' ]
[',' identifier2...]
```

This is, by far, the most complex construct in all of JAL, so I'll describe it one piece at a time below. Once variable definition is understood, everything else is easy!

VAR

Denotes the beginning of a variable definition.

VOLATILE

The VOLATILE keyword guarantees that a variable that is either used or assigned will not be optimized away, and the variable will be only read (or written) once when evaluating an expression.

Normally, if a variable is assigned a value that is never used, the assignment is removed and the variable is not allocated any space. If the assignment is an expression, the expression *will* be fully evaluated. If a variable is used, but never assigned, all instances of the variable will be replaced with the constant 0 (of the appropriate type) and the variable will not be allocated any space.

SHARED

Tells the compiler that this variable exists in shared memory, so there is no need to set bank bits (14 bit cores), or the BSR register (16 bit cores).

type[**cexpr]*

type is one of the predefined types (above). If type is BIT, BYTE, or SBYTE it can be extended using [**cexpr*]. For BYTE and SBYTE, this means the variable will be defined as an integer using *cexpr* bytes, e.g. WORD is simply shorthand for BYTE*2.

If type is BIT, the definition changes. A BIT variable, as defined in JAL, is really of type boolean. When assigned any non-zero value, it takes on the value of 1. Using the [**cexpr*], the definition changes to be more like a C bit field: assignment is masked. For example:

```
VAR BIT*2 cc
```

when assigning to cc, the assignment is:

```
cc = (value & 0x03)
```

identifier

Any valid JAL identifier

'[' [*cexpr*] ']'

Defines an arrayof *cexpr* elements. The array index starts at 0 and continues through (*cexpr* - 1). *cexpr* must be >= 1. An array *must* fit entirely within a single PIC data bank.

If *cexpr* is ommitted, the '=' term must exist and the size of the array will be set to the number of initializers present.

BIT arrays are *not* supported.

AT *cexpr* [':' bit]

Places the new variable at location *cexpr*. If it is a BIT variable, [':' *bit*] defines the bit offset with the location. Any location uses for explicit placement will not be allocated to another variable.

```
VAR BYTE fixed_variable at 0x20
```

AT *var* [':' *bit*]

Places the new variable at the same location as an existing variable. If it is a BIT variable, [':' *bit*] defines the bit offset with the location.

```
VAR BYTE data
VAR BIT data_bit at data:0 -- Overlay

VAR WORD channel_word
VAR BYTE channel_byte[2] at channel_word -- Overlay
```

Note that channel_byte[0] is the least significant byte and channel_byte[1] is the most significant byte of channel_word, which is a little-endian system. Because of this system these constructions also work:

```
VAR DOWRD base_data
VAR BYTE low_byte at base_data+0
VAR BYTE high_byte at base_data+1
VAR WORD upper_word at base_data+2
```

AT '{' *cexpr*1[',' *cexpr*2...] '}'

Places the new variable at multiple locations. On the PIC, many of the special purpose registers are mirrored in two or more data banks. Telling the compiler which locations hold the variable allows it to optimize the data access bits.

IS *var*

Tells the compiler that this identifier is simply an alias for another. This has been deprecated, use "ALIAS identifier IS identifier1" instead.

'=' *expr*

Shorthand assignment. The variable will be assigned *expr*.

'=' '{' *expr*1 [',' *expr*2...] '}'

For an array variable, the elements will be assigned *expr*1, *expr*2, ...

'=' "" ... ""

For a variable array, this assigns each ASCII value between "" and "" to one element of the constant array. Unlike C, there is no terminating NUL.

'=' "..."

For an array variable, the elements will be assigned one the ASCII values inside the quotes.

= "abc" is equivalent to = {"a", "b", "c"}

',' *identifier*2...

Allows defining multiple variables with the same attributes:

```
VAR BYTE a,b,c
```

Constants

Unnamed Constants

An unnamed numeric constant has the type UNIVERSAL, which is a 32-bit signed value. When a value of type UNIVERSAL is used in an operation, it is converted to the type of the other operand.

An exception to the above is floating point constants have type FLOAT.

Numeric constants have the following formats:

12 -- decimal

0x12 -- hexadecimal

0b01 -- binary

0q01 -- octal

"a" -- ASCII

1.23 or 1.23e2 or 1.23e-2 -- FLOAT

An ASCII constant evaluates to the first character except when used to initialize a constant or variable array in which case each character is used as one entry.

Example:

```
CONST ZERO = 0
CONST ZERO_CHAR = "0"
CONST ABC_ARRAY[] = "ABC"
```

The full format of a floating point constant is:

[+|-]###.[###[e[+|-]###]

Example:

```
VAR BYTE ch = "123"    ; ch is set to '1'
VAR BYTE str[] = "123" ; str[0] is set to '1'
                       ; str[1] is set to '2'
                       ; str[2] is set to '3'
```

Table 3: ASCII Constant Escaping

Sequence	Value
"\ooo"	octal constant
"\a"	bell
"\b"	backspace
"\f"	form feed
"\n"	line feed
"\qooo"	octal constant
"\r"	carriage return
"\t"	horizontal tab
"\v"	vertical tab
"\xdd"	hexadecimal constant
"\zbbb"	binary constant
"\\"	A single '\'

Constants other than ASCII constants may also contain any number of underscores ("_") which are ignored, but are useful for grouping. For example: 0b0000_1111

Named Constants

The complete format for defining a named constant is:

```
CONST [type[*cexpr]] identifier [ '[' [ cexpr ] ']' ]
   '=' { cexpr | '{' cexpr1[',' cexpr2...]'}' | '"'...'"'}
   [ ',' identifier2...]
```

CONST

CONST denotes the beginning of a constant definition clause.

type[*cexpr]

Defines the type of the constant. If none is given, the constant becomes universal type which is 32 bit signed.

'[' [*cexpr*] ']'

Defines a constant array (see array variable types). A constant array will not take any space unless it is indexed at least once with a non-constant subscript. On the PIC, constant arrays consume *code* space, not *data* space.

If *cexpr* is ommitted, the size of the array will be determined by the number of initializers used.

'=' *cexpr*

For non-array constants this assigns the value to the constant.

'=' '{' *cexpr*[',' *cexpr*2...] '}'

For arrays of constants this assigns the value to each element. There must be the same number of *cexprs* as there are elements defined.

'=' '"' ... '"'

For an array of constants, this assigns each ASCII value between '"' and '"' to one element of the constant array. Unlike C, there is no terminating NUL.

String Literals

String literals are enclosed in quotation markes '"..."' and can be used where ever an array of characters is allowed. The ASCII constant escaping noted under, `Unnamed Constants,' applies to each character within the string literal.

Note that a string literal terminates with the first NUL characters (0x00).

Aliases

Aliases allow a multiple identifiers (variables, named constants, sub-programs) to refer to the same object.

The format for defining an alias is:

```
ALIAS identifier IS identifier2
```

Often it is useful to allow a variable or constant be referred to by multiple names. For example, if on a certain project pin_a1 is a red LED, you might prefer to refer to it as RED_LED. That way if, on a different project pin_a2 is the red LED, you'd need only change the alias and everything else would continue to work fine.

```
ALIAS led is pin_a1
```

Chapter 4

Strings

JAL has two string types. A fixed-capacity string reserves a known number of bytes in data memory and needs no run-time support. A dynamic string is a handle into a compiler-managed heap and may grow and shrink at run time. Both are described in this chapter.

String literals are described with the constants in Constants on page 15, and the text conversion casts (STRING, HEX, BIN) and COUNT are described in Text conversion and COUNT on page 25.

Fixed-capacity strings

A fixed-capacity string stores up to N characters in N bytes of data memory. It needs no heap and can be used on every target.

```
VAR STRING[16] name = "text"
VAR STRING[8] tag
```

The bracketed value is the capacity in bytes and must be a constant expression. A shorter initializer or assignment is zero-filled up to the full capacity, so the first example's name holds "text" followed by twelve zero bytes; the characters can be inspected with the ordinary subscript syntax.

```
VAR STRING[16] name = "abc"
VAR BYTE first = name[0]          -- 'a'
VAR BYTE after = name[3]          -- 0
```

COUNT on a fixed-capacity string is the declared capacity, not the current text length: COUNT(name) is 16 above, whatever the contents. Use the length returned by the library, or the dynamic string type described in Dynamic strings and the string heap on page 20, when the length must vary at run time.

A value that is too long behaves differently for the two write forms. Assigning a string that does not fit copies as much as fits and sets _string_error to string_error_overflow. A concatenation whose result does not fit leaves the destination unchanged and sets _string_error to string_error_overflow. The error byte is described in Dynamic strings and the string heap on page 20.

Fixed-capacity strings may be passed to procedures and functions with matching capacity, and string literals are accepted as actuals for such formals: the literal is staged into a compiler string cell, zero-padded to the capacity, and any length up to the capacity compiles. A longer literal is a compile-time error that names the two lengths.

```
PROCEDURE send(STRING[16] IN s) IS ... END PROCEDURE

send("hello")                       -- staged, zero-padded
send("0123456789ABCDEF")            -- exactly 16 characters
```

Note: Fixed-capacity strings are not heap objects, so they never allocate or fail for lack of heap space. A too-long value is reported through _string_error as described above.

Dynamic strings and the string heap

A dynamic string, declared with VAR STRING and no capacity, is a handle to storage in a compiler-managed heap. Its contents may grow, shrink, and be concatenated at run time. Dynamic strings are supported on every target, but they need a heap whose size is fixed at compile time.

The heap size is declared with a WORD constant named _string_heap_size. It must appear before the first dynamic string and before the strings_dynamic library is included, because that library declares wrappers with STRING formals:

```
CONST WORD _string_heap_size = 255
INCLUDE strings_dynamic

VAR STRING name = "JALv3"
VAR STRING out
```

Each handle holds three index-sized fields (offset, length, and capacity) plus the payload bytes, which live in the heap. The width of the index fields is selected with a pragma (see below).

Heap configuration

```
PRAGMA STRING INDEX AUTO | BYTE | WORD
PRAGMA STRING ALLOC FIRST_FIT | BEST_FIT
PRAGMA STRING RETRY ON | OFF
```

INDEX selects the width of the per-handle offset, length, and capacity fields and of the heap block headers. AUTO chooses a width from the declared heap size; BYTE and WORD force the width.

ALLOC selects the free-block search: FIRST_FIT takes the first block that is large enough, BEST_FIT the smallest such block.

RETRY controls what happens when an allocation fails. With ON, the allocator compacts the heap once and retries the search; with OFF it fails immediately and sets the error byte.

The older PRAGMA STRING HEAP <bytes> line is still accepted, but it is ignored: it neither sets the heap size nor produces a warning. Use the _string_heap_size constant instead.

Assignment, copying, and concatenation

Assigning a literal allocates or resizes the payload. Assigning one dynamic string to another copies its contents. The & operator concatenates string values on the right-hand side of an assignment:

```
VAR STRING msg
VAR STRING dst
VAR STRING src = "world"

msg = "hello"
dst = src
msg = "Hello, " & src & "!"
msg = msg & ""                    -- empty by concatenation; frees the
 payload
```

Concatenation also works in expression position: as either operand of == or !=, and as a procedure argument. The chain is evaluated into a compiler-generated heap cell and then behaves like any other dynamic string.

```
IF msg == "Hello, " & src & "!" THEN ... END IF
IF "Hi, " & src == msg THEN ... END IF
print_string(serial_data, "t=" & src)
```

The & operator remains the bitwise AND when the left operand is not a string, so conditions such as (c1 == 1) & (c2 == 2) keep their meaning. Once a string operand has been seen, a non-string operand after & is a compile error.

Numbers in concatenations

Integer operands, constants and variables alike, render as ASCII decimal inside a & chain. All integer widths are supported, signed and unsigned, with a leading minus for negative values. Hexadecimal literals render in decimal too: 0x10 becomes "16".

```
VAR BYTE cnt = 42
VAR DWORD addr = 4294967295
VAR SQWORD big = -1234567890

out = "cnt=" & cnt & " addr=" & addr
out = "neg: " & -42 & " sqw: " & big
```

Floating point constants render as shortest round-trip decimal, so 1.5 becomes "1.5", 100.0 becomes "100", and 1e-8 becomes "0.00000001". Runtime floating point variables and BIT values are not converted and produce a clean compile error.

Length and errors

COUNT returns the current character count of a dynamic string, which is not the same as the capacity of a fixed-capacity string. The _length(expr) spelling is still accepted; with -warn-misc the compiler suggests COUNT instead.

The shared byte _string_error records the result of the most recent string operation. Reading it returns the stored code and resets the byte to none, so read it once per operation when the exact code matters. The symbolic constants are string_error_none, string_error_heap_full, string_error_overflow, and string_error_invalid.

```
VAR BYTE err = _string_error
IF err == string_error_heap_full THEN ... END IF
```

An assignment that fails for lack of heap space leaves the destination string unchanged and records string_error_heap_full.

Library helpers

After INCLUDE strings_dynamic the following functions are available. They return a BIT: TRUE on success, FALSE on failure or truncation.

```
string_append(string IN OUT dst, STRING IN src)
string_append_truncate(string IN OUT dst, STRING IN src, WORD IN max_len)
string_truncate(STRING IN OUT dst, WORD IN max_len)
string_from_array(STRING IN OUT dst, BYTE IN src_array[])
string_from_array_truncate(STRING IN OUT dst, BYTE IN src_array[], WORD IN
 max_len)
string_to_array(BYTE OUT dst_array[], STRING IN src)
string_to_array_truncate(BYTE OUT dst_array[], STRING IN src, WORD IN
 max_len)
string_heap_compact()
```

The truncating helpers return FALSE when bytes were dropped. The same built-ins may also be spelled with a leading underscore, for example _string_append and _string_heap_compact.

string_heap_compact() packs the live heap blocks and patches every dynamic handle's offset. It returns FALSE when the work table cannot be emitted or heap support is missing.

Pointers to dynamic strings

A pointer whose pointee contract is VAR STRING can be bound to a dynamic string, and assignments and concatenations through the pointer operate on the string it refers to. See Pointers on page 39.

```
VAR POINTER p AS VAR STRING
p IS name
p = "hello"
p = p & " tail"
```

Chapter 5

Operators, Casting, Expressions

Operators

Table 4: JALv3 Operators

Operator	Operation	Result
COUNT	returns the number of elements in an array	UNIVERSAL
WHEREIS	return the location of an identifier	UNIVERSAL[6]
DEFINED	determines if an identifier exists	BIT
'(' expr ')’	Grouping	Result of evaluating expr
'-'[3]	Unary - (negation)	Same as operand
'+'[3]	Unary + (no-op)	Same as operand
'!'	1's complement	Same as operand
'!!'[3]	Logical. If the following value is 0, the result is 0, otherwise the result is 1	BIT
'*'[3 5]	Multiplication	Promotion[2]
'/'[3 5]	Division	Promotion[2]
'%'[5]	Modulus division (remainder)	Promotion[2]
'+'[3]	Addition	Promotion[2]
'-'[3]	Subtraction	Promotion[2]
'<<'	Shift left	Promotion[2]
'>>'[1]	Shift right	Promotion[2]
'<'[3]	Strictly less than	BIT
'<='[3]	Less or equal	BIT
'=='[4]	Equality	BIT
'!='[4]	Unequal	BIT
'>='[3]	Greater or equal	BIT
'>'[3]	Strictly greater than	BIT

'&'	Binary AND	Promotion[2]
'|'	Binary OR	Promotion[2]
'^'	Binary exclusive OR	Promotion[2]

[1]shift right: If the left operand is signed, the shift is arithmetic (sign preserving). If unsigned, it is a simple binary shift.

[2]promotion: The promotion rules are tricky, here are the cases:

- If either operand is FLOAT, the result is FLOAT.
- If one of the operands is UNIVERSAL and the other is not, the result is the same as the non-UNIVERSAL operand.
- If both operands have the same signedness and width, the result is that of the operands.
- If both operands have the same width, and one is unsigned, the result is unsigned.
- If one operand is wider than the other, the other operand will be promoted to the wider type.

[3]These operators allow FLOAT types.

[4]Floating point numbers should never be compared for equality due to the imprecise way in which they are stored. Attempting to do so will result in a warning from the compiler. Two different operations which should yield an identical mathematical result may compare unequal. The correct way to compare two floating point numbers, say A and B, is `abs((A - B)/B) < 1e-6' (floating point values have a nominal precision of 6 - 9 digits).

[5]Keep in mind that multiplication and division, even between integer types are very expensive operations in both code size and data size (see Chapter: Built-in Functions).

[6]The result of WHEREIS depends upon the identifier used:

- A procedure or function will return the CODE address of the entry point.
- An assembly label will return the CODE address of the label.
- A variable will return the DATA address of the variable.
- A simple constant or an unused constant array will generate an error.
- A constant array depends upon the processor family:
  - 16-bit: returns the CODE address of the data.
  - non-16 bit: returns the CODE address of the entry point to the lookup function.

Casting

Casting is the operation of changing the type of a value. This can be necessary for a number of reasons: when assigning a larger value to a smaller one, say a WORD to a BYTE, the compiler will issue a warning. An explicit cast will eliminate that warning:

```
VAR WORD xx
VAR BYTE yy
;
; the following assignment will issue:
; warning: assignment to smaller type; truncation possible
;
yy = xx
;
; no warning will be generated below
;
yy = BYTE(xx)
```

In the first case, the compiler wants you to know there might be an issue (a rather common one). In the second case, you've explicitly told the compiler you know these types are different, but that is OK.

Another case where casting is necessary is to guarantee correct promotion during an operation. Take the following:

```
VAR WORD xx
VAR BYTE yy
;
; this is not likely to do what you expect
;
xx = yy * yy
;
; this will generate the correct result
;
xx = WORD(yy) * WORD(yy)
```

Remember that an operator only sees its two operands, it has no other context. Say the value of yy is 255. In the first case xx will be assigned a value of 1: the lower eight bits of the result. In the second case, the value of yy is promoted to a WORD, so xx will be assigned 65025 which is more likely what you would expect.

Text conversion and COUNT

JALv3 provides three text conversion casts and the COUNT operator. Both are ordinary operators of the language: they may be used wherever a value is expected, including comparisons, concatenations, and procedure arguments.

Text conversion casts

STRING(x) renders an integer in decimal, or copies the bytes of a byte array as characters. HEX(x) renders a value in lowercase hexadecimal without a prefix, and BIN(x) does the same in binary. For both, an optional second argument sets a minimum number of digits, so HEX(26, 4) is "001a" and BIN(5, 8) is "00000101"; a value that needs more digits renders at its full width. For arrays, each element is rendered at its natural width, and HEX zero-pads each byte element to two digits.

```
CONST STRING S1 = string(26)       -- "26"
CONST STRING S2 = hex(26)          -- "1a"
CONST STRING S3 = hex(26, 4)       -- "001a"
CONST STRING S4 = bin(5)           -- "101"
CONST STRING S5 = bin(5, 8)        -- "00000101"
CONST BYTE TBL[3] = {1, 2, 3}
CONST STRING S6 = string(TBL)      -- the three bytes as characters
CONST STRING S7 = hex(TBL)         -- "010203"
```

Constant operands are folded into read-only memory at compile time, so the examples above need no heap. Runtime operands are rendered when the program runs, through the string machinery described in Dynamic strings and the string heap on page 20, and therefore need the string heap:

```
CONST WORD _string_heap_size = 64
VAR STRING out
VAR BYTE cnt = 26

out = "hex=" & hex(cnt, 4)       -- "hex=001a"
IF string(cnt) == "26" THEN ... END IF
```

The result is an ordinary string value: it can be compared with literals and composed in concatenation chains. The names STRING, HEX, and BIN followed by an opening parenthesis always denote the cast, even if a symbol of the same name exists.

COUNT

COUNT reports the size of its operand. What is counted depends on the kind of operand:

Table 5: COUNT results

Operand	COUNT returns
array variable, or fixed-size array formal	the number of elements
open-array formal	the number of elements passed by the caller
dynamic string	the current number of characters
fixed-capacity string[N]	the capacity N
scalar variable or constant	the storage width in bytes

```
VAR BYTE arr[10]
VAR QWORD wide
VAR STRING msg = "hello"

COUNT(arr)       -- 10
COUNT(wide)      -- 8
COUNT(msg)       -- 5 (current length)
COUNT(arr[0])    -- 1 (a byte)
```

COUNT is computed at compile time whenever the operand is static, and at run time for dynamic strings and open-array formals. The older _length(expr) operator is still accepted; with -warn-misc the compiler suggests COUNT instead.

Expressions

An expression is simply values (variable or constant) and operators. For example:

```
y = x
y = x + y
y = -x - y
y = (5 + (3 - 2x)) / z
```

Please take time to look at the operator and casting sections, as many bug reports have been generated by a misunderstanding.

Like C, but unlike Pascal, variables of different types can be mixed freely in an expression. In this case, the promotion rules listed under "operators" are in effect.

Chapter 6

Flow Control

BLOCK

Syntax:

```
BLOCK
   statement_block
END BLOCK
```

Creates a new block. Any variables defined in this block go out of scope at the block. Mainly useful with the CASE statement (below).

CASE

Syntax:

```
CASE expr OF
   cexpr1[',' cexpr1a...] ':' statement
   [ cexpr2[',' cexpr2a...] ':' statement ]
   [ OTHERWISE statement ]
END CASE
```

expr is evaluated and compared against each *cexpr* listed. If a match occurs, the *statement* to the right of the matching *cexpr* is executed. If no match occurs, the *statement* after OTHERWISE is executed. If there is no OTHERWISE, control continues after END CASE. Unlike Pascal, the behavior is completely defined if there is no matching expression.

Unlike C (but like Pascal) there is no explicit break. After a statement is processed, control proceeds past the END CASE.

Each *cexpr* must evaluate to a unique value.

Example:

```
CASE xx OF
   1:     yy = 3
   2,5,7: yy = 4
   10:    BLOCK
             yy = 5
             zz = 6
          END BLOCK
   OTHERWISE zz = 0
END CASE
```

Note that only one *statement* is allowed in each case, thus the reason for BLOCK as BLOCK...END BLOCK is considered a single *statement*.

FOR

Syntax:

```
FOR expr [ USING var ] LOOP
   statement_block
   [ EXIT LOOP ]
END LOOP
```

statement_block is executed expr times. If USING *var* is defined, the index is kept in *var*, beginning with zero and incrementing towards *expr*. If *var* is not large enough to hold *expr*, a warning is generated. If [EXIT LOOP] is used, the loop is immediately exited.

Note:

expr is evaluated once on entry to the FOR statement.

On normal exit, *var* is equal to expr. After, `EXIT LOOP,' *var* holds whatever value it had at the beginning of the loop.

If *expr* is a *cexpr* and is one larger than *var* can hold, the loop will be exited when *var* rolls over to zero. In this case, on exit *var* will be zero.

Example:

```
VAR BYTE n
FOR 256 USING n LOOP
   ...
END LOOP
```

On exit, n will be zero.

```
xx = 0
FOR 10 LOOP
   xx = xx + 1
   IF (xx == 5) THEN
      EXIT LOOP
   END IF
END LOOP
```

FOREVER

Syntax:

```
FOREVER LOOP
   statement_block
   [ EXIT LOOP ]
END LOOP
```

statement_block is executed forever unless [EXIT LOOP] is encountered, in which case the loop is immediately terminated. This is commonly used for the main loop in a program because an embedded program like this never ends.

Example:

```
xx = 5
```

```
yy = 6
FOREVER LOOP
   READ_ADC()
   CHANGE_SPEED()
   IF (speed == 5) THEN
      EXIT LOOP
   END IF
END LOOP
```

IF

Syntax:

```
IF lexpr THEN
   statement_block
[ ELSIF lexpr2 THEN
   statement_block ]
[ ELSE
   statement_block ]
END IF
```

This creates a test, or series of tests. The *statement_block* under the first *lexpr* that evaluates to 1 will be executed. Any number of ELSIF clauses are allowed. If no *lexpr* evaluates to true and the ELSE clause exists, the *statement_block* for the ELSE clause will be executed.

A special case of the IF statement is when any *lexpr* is a constant 0. In this case, the statement block is not parsed. This can be used for block comments.

```
IF 0
   this is a dummy block that won't even be parsed!
END IF
```

Example:

```
IF x == 5 THEN
   y = 7
ELSIF x == 6 THEN
   y = 12
ELSE
   y = 0
END IF
```

Repeat

Syntax:

```
REPEAT
   statement_block
   [ EXIT LOOP ]
UNTIL lexpr
```

statement_block will be executed until *lexpr* evaluates to 1, or until [EXIT LOOP] is encountered.

Example:

```
REPEAT
   xx = READ_ADC
```

```
UNTIL (xx < 5)
```

WHILE

Syntax:

```
WHILE lexpr LOOP
   statement_block
   [ EXIT LOOP ]
END LOOP
```

statement_block will be executed as long as *lexpr* evaluates to a 1, or until [EXIT LOOP] is encountered. This is similar to REPEAT above, the difference being the *statement_block* of REPEAT loop will always execute at least once, whereas that of a WHILE loop may never execute (because the test is done first).

Example:

```
WHILE no_button LOOP
   xx = xx + 1
   IF (xx == 10) THEN
      EXIT LOOP
   END IF
END LOOP
```

Chapter 7

Other Keywords

Assert

Format:

```
ASSERT expr
```

This is only useful if the "-emu" compiler option has been used, otherwise it is ignored. If *expr* results in a zero value, the emulator will stop at this point.

INCLUDE

Format:

```
INCLUDE filename
```

This instructs the compiler to stop parsing the current file, open and completely parse the include file, the return to this file on the next line. Note the included file must have an extension of '.jal' and the filename may not begin or end with a space.

Note that it is not possible to include the same file multiple times. Once a file is included, it will not be included again. Also be aware that the filename is taken literally – no transform is done on it. This should be taken into consideration if you are writing a library as some filesystems are case-sensitive, and others are not, so "MYLIBRARY" and "mylibrary" might be two different files.

Example:

```
INCLUDE 16f877
```

Message generating

The following keywords generate a message, just as if it came directly from the compiler. Each is followed by a string which will be displayed as part of the message.

_DEBUG

Format:

```
_DEBUG '"' ... '"'
```

Generates a debug message. This will only be seen if the"-debug" compiler option has been used.

Example:

```
_DEBUG "this file is being deprecated"
```

_ERROR

Format:

```
_ERROR '"' ... '"'
```

Generates an error message.

Example:

```
_ERROR "this function should not be used"
```

_WARN

Format:

```
_WARN '"' ... '"'
```

Generates a warning message.

Example:

```
IF !DEFINED(foo) THEN
   _WARN "foo is not defined"
END IF
```

Chapter 8

Sub-programs: Procedures and Functions

Syntax:

```
PROCEDURE identifier [ '(' [VOLATILE] type { IN | OUT |
  IN OUT } identifier2 [',' ...] ')' IS
   statement_block
END PROCEDURE
```

```
FUNCTION identifier [ '(' [VOLATILE] type { IN | OUT | IN OUT }
  identifier2 [',' ...] ')' RETURN type IS
   statement_block
END FUNCTION
```

The only difference between a PROCEDURE and a FUNCTION, is the former does not return a value, while the later does. The procedure *identifier* exists in the block in which the procedure is defined. A new block is immediately opened, and all parameters exist in that block. A parameter marked IN will be assigned the value passed when called. A parameter marked OUT will assign the resulting value to parameter passed when called. While in a sub-program, a new keyword is introduced:

```
RETURN [ expr ]
```

When executed, the sub program immediately returns. If the sub program is a FUNCTION, *expr* is required. If it is a PROCEDURE, *expr* is forbidden.

A sub-program is executed simply by using its name. If parameters are specified in the sub-program definition, all parameters are required, otherwise none are allowed, except for trailing parameters declared OPTIONAL (see Optional parameters and defaults on page 34). A FUNCTION can be used anywhere a value is required (in expressions, as parameters to other sub-programs, etc). There is no limit to the number of parameters.

The return value of a function call may be ignored. Calling a function as a statement is legal and discards the result silently, without an error or a warning; this applies to direct calls, pointer calls, and pseudo-variable 'get calls. The value is still computed and marshalled, so ignoring it costs nothing extra. This differs from the original jalv2 compiler, which warned return value ignored.

JALv3 is a pass by value language. Conceptually, an IN parameter is read once when the sub-program enters, and an OUT parameter is written once when the sub-program returns. This is not always desired. For example if a sub-program writes a string of characters to the serial port (passed as parameter), only the last character written will be sent. For this case we need VOLATILE parameters. These are either read each time used (IN) or written each time assigned (OUT). This is accomplished using pseudo variables (see below). If the value passed is not a pseudo-variable, a suitable one is created.

There are two ways to pass an array into a sub-program:

```
PROCEDURE string_write (BYTE IN str[5]) IS...
PROCEDURE string_write (BYTE IN str[]) IS...
```

The first follows the pass-by-value semantics noted above. An array variable of size 5, str, is allocated in the namespace of the procedure. Any callers must call with an array of exactly 5 bytes, which is copied into the local variable and used.

Alternately, the second version created a flexible array. This is pass-by-reference which means (1) the amount of data space used for str is only two or three bytes, and (2) any sized array can be passed in. This is generally far more useful, and far less wasteful. The operator COUNT can be used to determine the size of the array passed in.

Procedures and functions can be nested.

Example:

```
          FUNCTION square_root (WORD IN n) RETURN WORD IS

             WORD result
             WORD ix

             ix = 1
             WHILE ix < n LOOP
                n = n - ix
                result = result + 1
                ix = ix + 2
             END LOOP
             RETURN result

          END FUNCTION

          xx = square_root(xx)
```

Recursion is fully supported but due to the overhead it is discouraged.

Optional parameters and defaults

JALv3 procedures and functions may declare trailing parameters as optional and give them a default value. A call may then omit any trailing run of those arguments.

```
PROCEDURE blink(BYTE IN pin, OPTIONAL WORD IN delay_ms DEFAULT 500) IS
   ...
END PROCEDURE

blink(3)             -- delay_ms = 500
blink(3, 1000)       -- delay_ms = 1000
```

The rules are:

- OPTIONAL parameters must come after all required parameters. A required parameter after an optional one is an error.
- Only IN parameters may be optional. A default on an OUT or IN OUT parameter is an error.
- DEFAULT is mandatory and must be a constant expression: a literal, a constant, or constant arithmetic. The value is typed by the parameter.
- Optional parameters work with BIT, every integer width, VOLATILE formals, and open arrays whose default is a string literal.
- Direct calls only. Function-pointer calls keep the full-arity rule, and pseudo-variable routines ('get and 'put) do not support optional parameters at all.

An open array may declare a literal default. The literal is materialized once, and COUNT inside the callee sees its length:

```
PROCEDURE tag(OPTIONAL BYTE IN txt[] DEFAULT "n/a") IS
   print_byte_dec(serial_data, BYTE(COUNT(txt)))
```

```
END PROCEDURE

tag()                -- prints 3
tag("custom")        -- prints 6
```

A string literal passed to a STRING or STRING[N] formal is staged into a compiler string cell, so any length up to the capacity compiles. An optional string parameter may also use a string literal as its default.

The empty-string sentinel

When a call needs to supply a later optional argument while leaving an earlier one at its default, an empty string literal can stand in for the omitted value. Passing "" to a non-string optional parameter means omitted, and the declared default is used. The sentinel works in the middle of an argument list and for open-array defaults.

```
PROCEDURE draw(BYTE IN x, OPTIONAL BYTE IN colour DEFAULT 1,
    OPTIONAL BIT IN bold DEFAULT FALSE) IS ... END PROCEDURE

draw(10, "", TRUE)   -- colour defaults to 1, bold is TRUE
```

Only the empty literal is the sentinel. A string of two or more characters remains a type error, and a one-character literal keeps its long-standing char-code meaning, so "x" passes 120 to a byte parameter. Required parameters never accept the sentinel, and STRING-typed formals receive "" as a real empty string.

Optional arguments are filled in by the caller: the compiler emits an ordinary full-arity call, so the mechanism is identical on every target.

Chapter 9

Pseudo-variables

Syntax:

```
PROCEDURE identifier "'" PUT '(' type IN identifier2 ')' IS
   statement_block
END PROCEDURE
```

```
FUNCTION identifier "'" GET RETURN type IS
   statement_block
END FUNCTION
```

A pseudo-variable is a sub-program, or pair of sub-programs that work as if they are variables. If a 'PUT procedure is defined, any assignment to *identifier* is replaced by a call to the *identifier*'PUT procedure. Similarly, if a 'GET function is defined, any time the associated value is used is an implicit call to the function.

If both a 'GET and 'PUT sub-program are defined, the parameter type of the 'PUT must match the return type of the 'GET.

Example:

```
FUNCTION pin'GET() RETURN BIT IS
   return pin_shadow
END FUNCTION
```

```
PROCEDURE pin'PUT(BIT in xx) IS
   pin_shadow = xx
   port = port_shadow
END PROCEDURE
```

```
pin = 5
```

Chapter 10

Pointers

A pointer is a variable that holds the address of another object: a variable, an array, an array element, a constant, or a pseudo-variable. Pointers are new in JALv3 and are supported on every target. Every pointer declaration names the type of the object it may point at, the *pointee contract*.

Declaring a pointer

The preferred form puts the pointee contract after AS:

```
VAR POINTER p AS VAR BYTE          -- points at a writable byte
VAR POINTER cp AS CONST BYTE       -- points at a read-only byte
VAR POINTER ap AS VAR BYTE[5]      -- points at one five-byte array
VAR POINTER slots[3] AS VAR BYTE   -- three pointer slots
```

The older spellings are also accepted:

```
VAR POINTER VAR BYTE p
VAR POINTER CONST BYTE cp
VAR POINTER p(AS VAR BYTE)
```

A pointer occupies 2 bytes on the classic PIC cores, 3 bytes on PIC18, and 4 bytes on PIC32 and ESP32.

Binding a pointer

The IS statement makes a pointer refer to an object. It does not copy a value; it binds the address:

```
VAR BYTE value = 10
VAR POINTER p AS VAR BYTE

p IS value          -- p now points at value
```

The target may be a variable, an array, an array element, a constant array, a compatible pseudo-variable, or another pointer:

```
VAR BYTE arr[4]
VAR POINTER whole AS VAR BYTE[4]
VAR POINTER element AS VAR BYTE
VAR POINTER other AS VAR BYTE

whole IS arr        -- the array
element IS arr[2]   -- one element
other IS whole      -- copy the pointer value (not the pointee)
```

Binding a pointer array element works with a constant or a runtime index: slots[k] IS value.

Reading and writing through a pointer

Using a pointer as a value reads the pointee; assigning to it writes the pointee. Both require the pointer to have been bound.

```
VAR BYTE value = 10
VAR BYTE copy
VAR POINTER p AS VAR BYTE

p IS value
copy = p            -- copy = 10
p = 25              -- value = 25
```

With an array pointee, a subscript selects the element. Constant and runtime indices are both allowed:

```
VAR BYTE arr[4]
VAR POINTER ap AS VAR BYTE[4]

ap IS arr
ap[0] = 50          -- arr[0] = 50
copy = ap[i]        -- read element i
```

Note: Plain assignment between two pointers is a store-through, not a pointer copy: q = p reads the value p points at and writes it through q. Use q IS p to copy the pointer itself.

Read-only pointees

A pointee declared CONST may be read through the pointer but not written. Binding it to a constant array is the usual way to walk a ROM lookup table:

```
CONST BYTE table[3] = {10, 20, 30}
VAR POINTER cp AS CONST BYTE[3]

cp IS table
copy = cp[1]        -- 20
```

Pseudo-variables

A pointer may be bound to a pseudo-variable, in which case reads and writes go through its accessors: a read uses the 'get routine and a write uses the 'put routine. An indexed read of an array-shaped pseudo-variable (p[i]) uses its indexed accessor.

```
FUNCTION sensor'GET() RETURN BYTE IS ... END FUNCTION
PROCEDURE sensor'PUT(BYTE IN value) IS ... END PROCEDURE

VAR POINTER p AS VAR BYTE
p IS sensor
copy = p            -- calls sensor'GET
p = 5               -- calls sensor'PUT
```

Pseudo-variable bindings live on scalar pointer variables. They cannot be stored in pointer-slot array elements or record members, and a pointer bound to a pseudo-variable cannot be copied with q = p (the store-through form drops the accessor link); read the value into an ordinary variable first. A pseudo-variable binding is also not carried through procedure parameters: a pointer formal receives the address only.

Pointers to dynamic strings

A pointer whose pointee contract is VAR STRING points at a dynamic string. Assignment and concatenation through the pointer act on the string it refers to:

```
CONST WORD _string_heap_size = 128
INCLUDE strings_dynamic
VAR STRING text
VAR POINTER p AS VAR STRING

p IS text
p = "abc"
p = p & "XY"
```

Pointer slots and WHEREIS

An array of pointers holds several bindings. Elements are bound with the same IS statement, including at runtime indices, and read and written through the element:

```
VAR BYTE a = 0
VAR BYTE b = 0
VAR BYTE value
VAR POINTER slots[2] AS VAR BYTE

slots[0] IS a
slots[1] IS b
value = slots[1]    -- 0
slots[k] IS b       -- rebind slot k at run time
```

WHEREIS returns the address a pointer targets, or zero when the pointer is unbound. It accepts a pointer variable or an indexed slot:

```
VAR DWORD addr
addr = WHEREIS(slots[0])
addr = WHEREIS(a)
```

Checks

The compiler tracks whether each pointer has been bound and rejects reads, writes, subscripts, and self-binds of an unbound pointer. A pointer slot that has never been bound contains zero; indirect access through such a slot is refused at run time instead of reading address zero.

Procedure and function pointers

A pointer may also hold the address of a procedure or a function. Such a pointer declares the complete signature of the routine it can point at, and calls through it look like ordinary calls.

Declaring a callable pointer

```
VAR POINTER handler AS PROCEDURE (BYTE IN)
VAR POINTER setter  AS PROCEDURE (BYTE OUT)
VAR POINTER bumper  AS PROCEDURE (BYTE IN OUT)
VAR POINTER calc    AS FUNCTION (BYTE IN) RETURN WORD

VAR POINTER handlers[4] AS PROCEDURE (BYTE IN)
VAR POINTER calcs[4] AS FUNCTION (BYTE IN, BYTE IN) RETURN BYTE
```

The signature must match the routine exactly: the same number of parameters, the same parameter directions, and the same return type. A mismatch is a compile error.

Binding and calling

```
PROCEDURE log(BYTE IN value) IS ... END PROCEDURE
FUNCTION double(BYTE IN value) RETURN WORD IS ... END FUNCTION

VAR POINTER handler AS PROCEDURE (BYTE IN)
VAR POINTER calc AS FUNCTION (BYTE IN) RETURN WORD
VAR WORD result

handler IS log
calc IS double

handler(42)          -- calls log
result = calc(21)    -- calls double, result = 42
```

Every parameter direction works through a pointer call. An OUT or IN OUT actual is marshalled back into the caller's variable after the call returns, so the call site cannot tell a pointer call from a direct call:

```
PROCEDURE set(BYTE OUT value) IS value = 21 END PROCEDURE
PROCEDURE bump(BYTE IN OUT value) IS value = value + 1 END PROCEDURE

VAR POINTER sp AS PROCEDURE (BYTE OUT)
VAR POINTER bp AS PROCEDURE (BYTE IN OUT)
VAR BYTE victim
VAR BYTE counter = 25

sp IS set
sp(victim)           -- victim = 21

bp IS bump
bp(counter)          -- counter = 26
```

Pointer slots

An array of callable pointers holds several routines that share one signature. Slots are bound with IS and called with a constant or a runtime index:

```
PROCEDURE proc_c(BYTE IN test) IS ... END PROCEDURE
PROCEDURE proc_d(BYTE IN test) IS ... END PROCEDURE

VAR POINTER proc_slots[2] AS PROCEDURE (BYTE IN)
VAR BYTE step = 0

proc_slots[0] IS proc_c
proc_slots[1] IS proc_d

WHILE step < 2 LOOP
   proc_slots[step](0)      -- dispatch by runtime index
   step = step + 1
END LOOP
```

WHEREIS works on an indexed slot and reports the address of the routine it is bound to, or zero when the slot is unbound.

Return values

A scalar procedure pointer is called exactly like the procedure it points at. A function pointer is called wherever a value is expected. The return value of a function call may also be ignored: calling a function as a statement is legal and discards the result without an error or a warning. This applies to direct calls, pointer calls, and pseudo-variable 'get calls.

Chapter 11

Interrupts

Syntax:

```
PROCEDURE identifier IS PRAGMA INTERRUPT [FAST]
   statement_block
END PROCEDURE
```

PRAGMA INTERRUPT tells JAL that this procedure can only be called by the microcontroller's interrupt processing. Any number of procedures can be defined as an interrupt handler. When an interrupt occurs, first the microprocessor state is saved, then control passes to the first procedure marked as an interrupt handler. Control continues to pass to each interrupt handler until the last, then the microprocessor state is restored and the interrupt ended. The programmer is responsible for clearing whatever bits caused the interrupt to happen. A procedure marked as an interrupt handler cannot be called directly from elsewhere in the program. Beyond that, an interrupt handler can do anything any other procedure can do. The order the interrupt handlers are called is undefined, the only guarantee is each handler will be called at each interrupt, and will only be called once.

If an interrupt handler executes a sub-program that is also executed by the main body of the program, that sub-program will be marked recursive and incur the recursion overhead each time it is called.

If FAST is declared, the interrupt handler will only save the minimum amount of state necessary. This must be used with great care -- although the microprocessor state is saved, state used internally by the compiler is not. As such, only a completely assembly sub-program should be used. Any JAL statements might invalidate the internal state of the compiler. If any interrupt handler is marked FAST then only one interrupt handler is allowed.

Chapter 12

Tasks

Syntax:

```
TASK identifier '()' IS
   statement_block
END TASK
```

JALv3 introduces the concept of TASKs, which are a form of co-operative multi-tasking. Unlike preemptive multi-tasking, where control passes from one task to another automatically, control passes only when a task explicitly yields. Tasks take no parameters.

The number of task slots is set with the TASK pragma, or with the compiler option -task:

```
PRAGMA TASK cexpr
```

The count includes the main program itself. To run the main program plus three worker tasks, use PRAGMA TASK 4. It is generally better to set this in the program than with the compiler option, because the number is unlikely to change.

A task is started with the _task_start statement. It may be started from the main program or from another task:

```
_task_start worker_a()
```

A task yields the processor with _task_yield. The scheduler is a deterministic round robin: each yield hands control to the next live task, and a task that returns, or falls off the end of its body, removes itself from the schedule:

```
TASK worker_a() IS
   ...
   _task_yield
   ...
END TASK
```

The older SUSPEND keyword still works but is deprecated and produces a warning; use _task_yield instead.

Running tasks can be manipulated from the main program or from a task with these statements. Each names a TASK and accepts an optional empty parameter list:

```
_task_suspend identifier[()]
_task_resume  identifier[()]
_task_kill    identifier[()]
_task_restart identifier[()]
```

Only one copy of a task body may run at a time. Do not start a task that is already live.

Task local variables are allowed only on the 32-bit targets, where every automatic local receives a unique static address. On the PIC back ends all procedures share one software frame area, so a suspended task's locals could collide with another task's; declaring a variable inside a TASK on those targets is an error. Keep task state in globals there. On every target, a task body runs on its starter's stack, so keep the frames shallow.

Example:

```
PRAGMA TASK 3

VAR BYTE a_count = 0
VAR BYTE b_count = 0

TASK task_a() IS
   a_count = a_count + 1
   _task_yield
   a_count = a_count + 1
END TASK

TASK task_b() IS
   b_count = b_count + 1
   _task_yield
   b_count = b_count + 1
END TASK

_task_start task_a()
_task_start task_b()
_task_yield
_task_kill task_b()
```

Chapter 13

Assembly

When all else fails, one can resort to inline assembly. This can be in the form of a single statement:

```
ASM ...
```

or an entire block:

```
ASSEMBLER
   statements
END ASSEMBLER
```

Using assembly should be a last resort -- it is needed only when either a feature is not possible using JALv3 (for example, the TRIS and OPTION codes), or when speed is of the essence. JALv3 includes the entire assembly language set in the PIC16F87x data sheet, several instructions from earlier micro controllers, and several common macros. There is some support for the 16 bit keywords.

To guarantee the correct data bank is selected when accessing a file register, use one of the following:

```
BANK opcode ...
```

Or:

```
BANK f
```

The former takes the file register from the command, the later takes it directly.

Similarly, to guarantee the correct page bits are set (for GOTO or CALL), use one of the following:

```
PAGE opcode ...
```

Or:

```
PAGE lbl
```

Again, the former takes the label from the command, the later takes it directly.

Normally, the codes to set or clear the bank or page bits are only generated when necessary. If the bits are already in the correct states, no further commands are generated. If you need to guarantee the codes are always generated, use the following pragmas:

```
PRAGMA KEEP PAGE
PRAGMA KEEP BANK
```

The former will keep any page bits, the later and bank bits. These affect the entire sub-program in which they are declared.

To declare a local label for use in CALLs and/or GOTOs:

```
LOCAL identifier[',' identifier2...]
```

Once declared, a label is inserted into the assembly block by making it the first part of a statement, followed by a ':':

```
identifier: opcode...
```

The available opcodes are listed below. For a full description see the appropriate data sheet.

Note that when using inline assembly you should not modify the bank or page registers, FSR, or BSR. If these are modified, it is the programmers responsibility to return them to their original states.

Available Op-codes

The following abbreviations are used:

b -- bit number, 0 <= b <= 7

d -- destination, 'f' or 'w'

f -- file register or variable

n -- literal value, 0 <= n <= 255 unless otherwise noted

k -- label or constant

Note that not all opcodes are available on all devices. Check the datasheet for a complete description.

addwf f,d

addwfc f,d

andwf f,d

clrf f

clrw

comf f,d

decf f,d

decfsz f,d

incf f,d

incfsz f,d

iorwf f,d

movf f,d

movwf f

nop

rlf f,d

rlcf f,d

rlncf f,d

rrf f,d

rrcf f,d

rrncf f,d

subwf f,d

swapf f,d

xorwf f,d

bcf f,b

bsf f,b

btfsc f,b

btfss f,b

addlw n

andlw n

call k

clrwdt

goto k

iorlw n

movlw n

retfie

retlw n

return

sleep

sublw n

xorlw n

tblrd { * | *+ | *- | +* }

tblwt { * | *+ | *- | +* }

reset

option

tris n (5 <= n <= 9)

Common Macros

addcf f,d

adddcf f,d

b k

bc k

bdc k

bnc k

bndc k

bnz k

bz k

clrc

clrdc

clrz

lcall k

lgoto k

movfw f

negf f

setc

setdc

setz

skpc

skpdc

skpnc

skpndc

skpnz

skpz

subcf f,d

subdcf f,d

tstf f

Data Directives

The following allow data to be directly inserted into the code area. Retrieving these data is chip-specific. Also, as the data go directly into the program memory, the amount of space actually used is chip specific.

Below, the term *list* is a comma separated list of constants or strings.

db *list*

Inserts a list of bytes, one per program word.

dw *list*

Inserts a list of words. On 12 & 14 bit cores each word can be 14 bits (0..8191), whereas on 16 bit cores each word can be 16 bits (0..65535).

ds *list*

Pack two 7-bit values into a program word. Not necessary on the 16 bit cores.

Chapter 14

Built-in Functions

JALv3 attempts to be a minimal language with most complex operations done with sub-programs, however some functions simply cannot be efficiently supported externally.

Multiplication, Division, Modulus Division

Multiplication, Division, and Modulus Division are internal mainly because there is no way to predetermine the size of the operands. Note that unlike the other operators which are done inline, these are function calls and require one stack entry when used!

A second reason for having these built in is the optimizer -- when a multiplication or division by 1 is done, the operation is ignored. When a multiplication or division by a power of two is done, the resulting code is performed using shifts instead.

For both of these operations, the code generated will be that required for the largest operands unless -fastmath is passed to the compiler. For example, if the operation occurs only between two BYTEs, the 8-bit routine will be generated. If it occurs between BYTEs and WORDs, the 16-bit routine will be generated.

If -fastmath is used, a different function will be generated for each argument type.

The compiler keeps track of the last operation, so if you find yourself needing both the division result and the remainder of, a certain operation, make sure to put the assignments close together, thus saving a function call:

```
n = x / 10
r = x % 10
```

will only result in one call to the division -- the assignment to r will be a simple assignment.

Floating Point Operations

Most floating point operations are done via function calls because of the size of the code generated, so at least one stack entry will be used per operation. For multiplication and division, two stack entries are required because these rely on the integer routines.

Operations that do not require a function call include: multiplication or division by a power of 2, assignment from a constant.

JALv3 follows IEEE 754 single precision: results are rounded to nearest even, denormal values are represented, and infinity and NaN follow the standard. A floating point value occupies 4 bytes (see Types on page 9).

_usec_delay(*cexpr*);

_usec_delay(*cexpr*) is useful when an exact delay is required. It generates code that is guaranteed to delay a given number of micro-seconds. This is done using loops with one, two, or three variables, and no-op instructions as necessary.

For _usec_delay to work correctly, interrupts must be disabled, and `PRAGMA TARGET CLOCK' must be issued to set the system clock speed.

Note that _usec_delay() will generate delays up to 4,294.967295 seconds (or ~71.5 minutes), this isn't really the best use of space. On a 20MHz 16f877 this required 1043 instructions.

This is typically used for delays of a few 10s or 100s of uSec.

Chapter 15

Operator Precedence

JAL uses the following operator precedence.

1	() []	Function call, array/member access
2	! - + COUNT type cast	Unary operations, COUNT, type cast
3	* / %	Multiplication, division, modulo
4	+ -	Addition and subtraction
5	<< >>	Bitwise shift left, shift right
6	< <= > >=	Comparison: less-than, greater-than
7	== !=	Comparison: equal, not equal
8	&	Binary AND
9	^	Binary exclusive OR (XOR)
10	|	Binary inclusive (normal) OR
11	=	Assignment

Chapter 16

Limitations and known issues

The JAL compiler does not support all features of all PICs. When a specific feature is available, it should be disabled in your program by the corresponding `pragma target` if a pragma for it exist. These pragmas can be found at the end of the device file of the PIC under `pragma fusedef`.

In some cases a PIC has specific manufacturing features that do not work well when using a PIC in its application.

The following limitations and issues are known:

Table 6: Limitations and known issues

Feature	Pragma target	Setting	Remarks
Linear memory	-	-	Not supported by the compiler
Extended instruction set	XINST	DISABLED	Not supported by the compiler
Multi vectored interrupts (PIC18)	MVECEN	DISABLED	Not supported on PIC18; PIC32MX uses PRAGMA INTERRUPT MVEC
JTAG boundary scan	JTAGEN	DISABLED	To enable all I/O pins